

2. Découverte de l'en-tête TCP

Pour chaque champ ou flag :

Quel est son nom ?

Qu'est-ce qu'un flag TCP ?

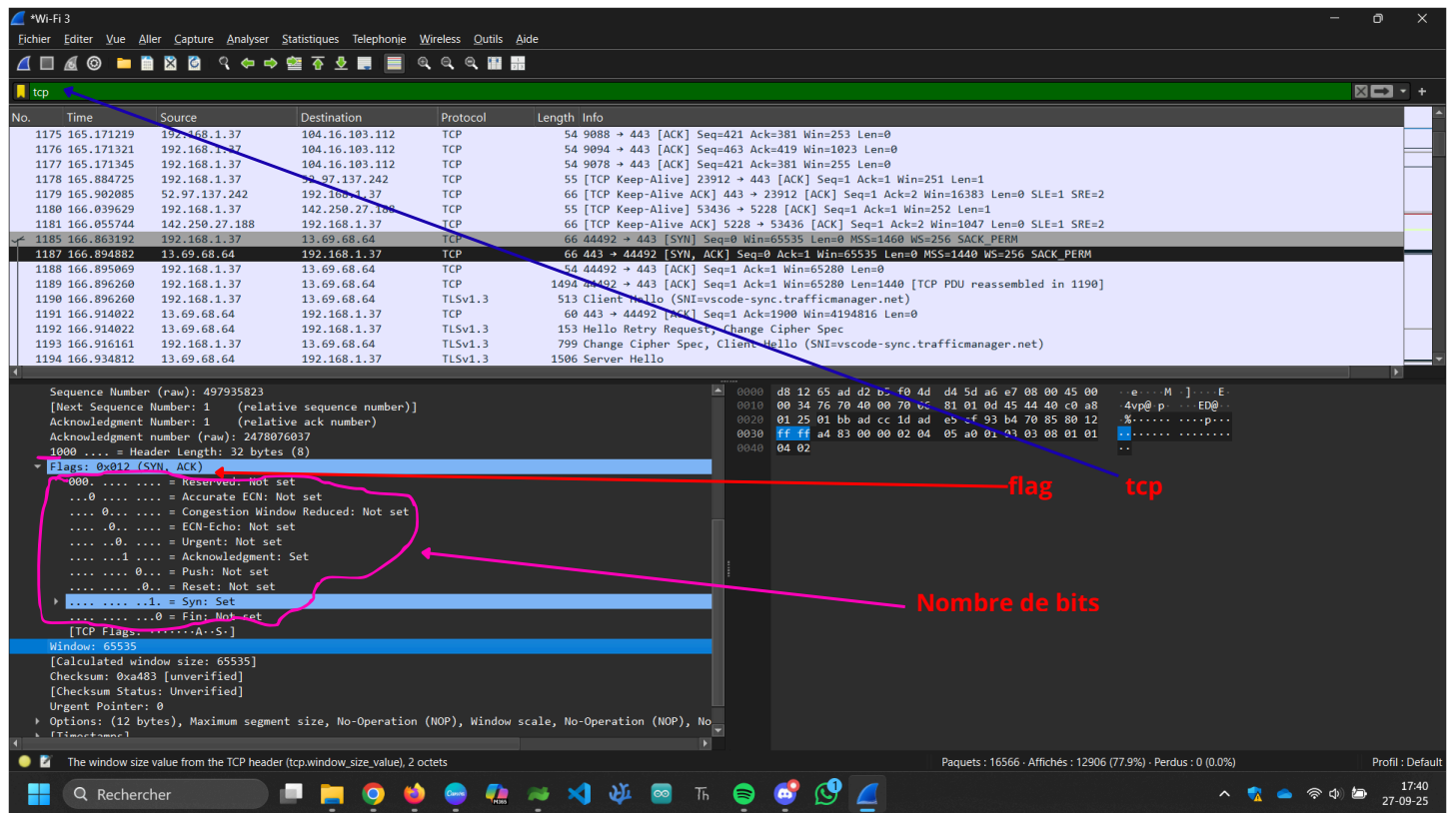
TCP (Transmission Control Protocol) est un protocole **de transport**. Pour gérer la communication, il utilise des **drapeaux (flags)** dans chaque segment TCP.

Les principaux flags sont :

Flag	Nom complet	Utilité
SYN	Synchronize	Initier une connexion
ACK	Acknowledgment	Confirmer la réception d'un segment
FIN	Finish	Terminer une connexion proprement
RST	Reset	Réinitialiser une connexion
PSH	Push	Indiquer que les données doivent être traitées immédiatement
URG	Urgent	Les données sont urgentes
ECE / CWR	Explicit Congestion / Congestion Window Reduced	Gestion de la congestion (rarement visible dans un TCP classique)

💡 Pour un TCP classique, les flags les plus courants sont **SYN, ACK, FIN et RST**.

Sur combien de bits est-il encodé ?



Trouve le champ Flags

- Dans l'en-tête TCP, tu verras quelque chose comme :

Flags: 0x012 (SYN, ACK)

- Clique sur la petite flèche à côté de **Flags** pour **déplier le détail des bits**.

Wireshark montre chaque bit

Quand tu développes **Flags**, tu verras quelque chose comme :

00010010 Flags: 0x012

0... .. = FIN: 0

.0.. .. = SYN: 1

..1. = RST: 0

...0 = PSH: 0

.... 1... = ACK: 1

.... .0.. = URG: 0

.... .0. = ECE: 0

.... ...0 = CWR: 0

- Chaque flag = **1 bit**
- 0 = flag désactivé, 1 = flag activé
- Wireshark te montre **la position exacte du bit dans le champ**

Interprétation

- Tu sais combien de bits sont utilisés : **1 bit par flag**
 - Tu vois quels flags sont activés (1) ou désactivés (0)
 - Tu peux aussi vérifier le code hex (0x012) et le convertir en binaire si tu veux comprendre “manuellement”.
-

Donnez quelques exemples de valeur pour ce champ

Tableau des flags TCP et valeurs possibles

Flag	Bit	Exemple 0 (désactivé)	Exemple 1 (activé)	Description courte
NS (Nonce Sum)	0	0	1	Flag expérimental, rarement utilisé
CWR (Congestion Window Reduced)	1	0	1	Indique une congestion réduite
ECE (ECN Echo)	2	0	1	Signale la congestion TCP
URG (Urgent)	3	0	1	Données urgentes
ACK (Acknowledgment)	4	0	1	Reconnaissance de réception
PSH (Push)	5	0	1	Pousse les données immédiatement
RST (Reset)	6	0	1	Réinitialise une connexion
SYN (Synchronize)	7	0	1	Début d'une connexion
FIN (Finish)	8	0	1	Fin d'une connexion
Reserved / autre	9-11	0	0 ou 1 selon extension	Réservé pour usage futur

Exemple concret de valeurs

a) Handshake TCP classique (SYN → SYN+ACK → ACK)

Paquet	Flags activés	Bits binaires (simplifiés)
Client → Serveur	SYN	0000001000
Serveur → Client	SYN, ACK	0001001000
Client → Serveur	ACK	0001000000

- Chaque 1 correspond à un flag activé, chaque 0 à un flag désactivé.
- Les bits à gauche représentent les flags plus « rares » (NS, CWR, ECE).

b) Terminer une connexion (FIN)

Paquet	Flags activés	Bits binaires
Client → Serveur	FIN, ACK	0001000100
Serveur → Client	ACK	0001000000

Astuce Wireshark

- Quand tu cliques sur **Flags**, Wireshark te montre :
 - **Code hex** (ex: 0x12)
 - **Binaire** (ex: 00010010)
 - **Flags activés en clair** (ex: SYN, ACK)

Avec ce tableau, tu peux **lire n'importe quel paquet TCP** et savoir quels flags sont activés.

Faites une hypothèse sur son usage

Validez/invalides cette hypothèse par une courte recherche Web

Reformulez en deux lignes la signification et l'usage de ce champ

3. Questions sur l'en-tête TCP

Quelle est la longueur de l'en-tête TCP ?

Définition simple

L'en-tête TCP (ou *TCP header*) est la partie de contrôle ajoutée au début de chaque segment TCP.

👉 C'est comme une "fiche d'identité" qui contient toutes les informations nécessaires pour :

- ouvrir une connexion,
- transférer correctement les données,
- vérifier l'intégrité,
- fermer la connexion.

Longueur minimale : 20 octets (160 bits) → quand il n'y a pas d'options TCP.

Longueur maximale : 60 octets (480 bits) → quand des options (comme MSS, Window Scale, Timestamps) sont présentes.

Le champ **Data Offset** (4 bits) de l'en-tête TCP indique cette longueur.

- Port 443 → HTTPS
- Port 21 → FTP
- Port 22 → SSH
- Port 25 → SMTP
- Port 53 → DNS (sur UDP et TCP)

Ces ports sont standardisés par l'IANA (Internet Assigned Numbers Authority).

Il existe trois catégories de numéros de ports. Expliquez.

Les 3 catégories de ports

Ports bien connus (Well-Known Ports)

- **Plage : 0 à 1023**
- Utilisés par les services système et les protocoles standards.
- Exemples :
 - 20/21 → FTP
 - 22 → SSH
 - 25 → SMTP
 - 53 → DNS
 - 80 → HTTP
 - 443 → HTTPS
- ⚠ Souvent réservés à l'administrateur (root) pour éviter des conflits.

Ports enregistrés (Registered Ports)

- **Plage : 1024 à 49151**
- Utilisés par des applications spécifiques (après enregistrement auprès de l'IANA).
- Exemples :
 - 3306 → MySQL
 - 5432 → PostgreSQL
 - 27017 → MongoDB
 - 8080 → HTTP alternatif
- 💡 Ces ports ne sont pas aussi critiques que les "bien connus", mais sont souvent associés à des logiciels précis.

Ports dynamiques ou privés (Dynamic / Private / Ephemeral Ports)

- **Plage : 49152 à 65535**
- Utilisés temporairement par les clients pour initier une connexion.
- Exemple :
 - Si ton PC ouvre une connexion HTTP (vers le port 80 du serveur), il utilisera un **port aléatoire dans cette plage** comme port source (ex : 50432).
- ⚡ Ces ports ne sont pas enregistrés → ils servent à de l'utilisation "temporaire/dynamique".

✓ Résumé simple

- **0-1023** : ports **réservés aux services système** (HTTP, DNS, SSH...).
 - **1024-49151** : ports **enregistrés pour des applis spécifiques** (MySQL, etc.).
 - **49152-65535** : ports **éphémères/dynamiques**, choisis par le client.
-

Que représente le numéro de séquence ?

Quand tu envoies des données avec **TCP**, ce n'est pas "les segments" qu'on numérote directement, mais **chaque octet** (chaque byte).

Exemple simple :

- Imagine que tu veux envoyer le message "**Bonjour**" (7 octets).
- TCP décide de le couper en deux segments :
 - Segment 1 : "Bon" (3 octets)
 - Segment 2 : "jour" (4 octets)

Maintenant, TCP attribue un **numéro de séquence** à chaque octet :

- Segment 1 → commence au numéro 1000
 - "B" = 1000
 - "o" = 1001
 - "n" = 1002
- Segment 2 → commence au numéro **1003** (car le dernier du premier segment était 1002)
 - "j" = 1003
 - "o" = 1004
 - "u" = 1005
 - "r" = 1006

👉 Donc, le **numéro de séquence** d'un segment = **le numéro du premier octet qu'il transporte**.

Pourquoi ?

- Le récepteur peut **remettre dans l'ordre** si les segments arrivent mélangés.
 - Il peut aussi **voir qu'il manque des octets** si une partie du flux ne correspond pas à la suite attendue.
-

Comment est-il incrémenté entre deux segments d'une même connexion ?

Attention : dans ses préférences, Wireshark a une option "numéros de séquence relatifs". En quoi consiste-t-elle? Activez ou désactivez-la pour observer la différence.

Le **numéro de séquence** TCP n'est pas incrémenté de **1 segment** → **1 segment**, mais de **nombre d'octets envoyés**.

Règle :

- Si un segment transporte **N octets**,
- Alors le segment suivant aura comme numéro de séquence = **(numéro de séquence précédent) + N**.

Exemple :

1. Segment A :
 - Numéro de séquence = **1000**
 - Taille = **500 octets**
 - Donc il couvre les octets **1000 → 1499**
2. Segment B :
 - Numéro de séquence = **1500** (car 1000 + 500)
 - Taille = **1000 octets**
 - Donc il couvre les octets **1500 → 2499**

👉 L'incrémentation dépend donc **de la quantité de données transportées**, pas du nombre de segments.

Cas particulier : segments sans données

- Certains segments TCP ne transportent pas de données (ex. : juste un **ACK** ou une ouverture de connexion avec **SYN**).
- Dans ce cas, le numéro de séquence **n'avance pas** (reste le même).
- Exception : le flag **SYN** et **FIN** consomment chacun **1 unité de séquence** même sans données (pour être pris en compte dans l'ordre).

Attention à Wireshark

Par défaut, les **numéros de séquence relatifs** sont activés.

- Ça veut dire qu'il commence à compter à **0** pour la première donnée (plus facile à lire).

Si tu désactives l'option, tu verras les vrais numéros envoyés par TCP (souvent des grands nombres aléatoires choisis au départ).

Dans **Wireshark**, tu peux activer/désactiver l'affichage des **numéros de séquence relatifs**.

Pour le faire :

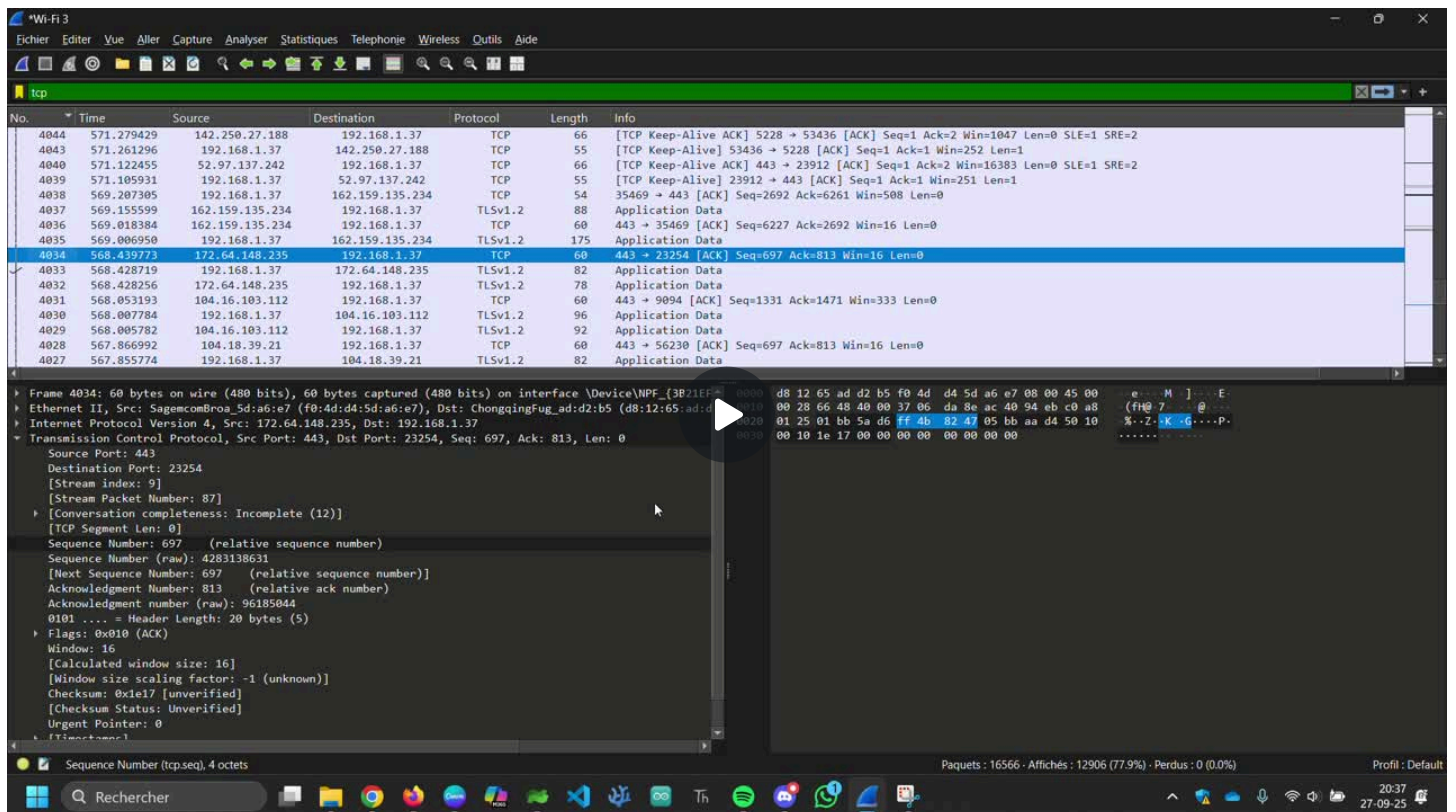
1. Ouvre **Wireshark**
2. Va dans le menu :
Édition (Edit) → Préférences (Preferences)
3. Dans la fenêtre des préférences, déroule :
Protocols → TCP
4. Décoche (ou coche) l'option :
"Relative sequence numbers"

👉 Quand c'est **coché** :

- Wireshark commence à compter les séquences à partir de **0** (plus lisible).

👉 Quand c'est **décoché** :

- Tu verras les **vrais numéros** utilisés sur le réseau, qui sont des grands nombres aléatoires (choisis pour chaque connexion afin d'éviter certaines attaques).



Que représente le champ "Acknowledgement Number" ?

Définition simple : Acknowledgment Number

- Le champ **"Acknowledgment Number"** (Numéro d'accusé de réception) indique :
le **numéro de séquence du prochain octet que l'émetteur attend de recevoir**.

- Autrement dit, c'est comme dire à l'autre machine :

"J'ai bien reçu tous les octets jusqu'au numéro N-1, j'attends maintenant l'octet numéro N."

Quel est le lien avec :

Le champ "Sequence number" (d'un autre segment à déterminer)

Le flag ACK (du même segment)

Lien avec le champ Sequence Number

- Chaque segment TCP a un **Sequence Number** : le numéro du **premier octet de données** dans ce segment.
- Exemple :
 - Si Sequence Number = 1000, ça veut dire que **l'octet 1000** est le premier octet de ce segment.
- Le **Acknowledgment Number** d'un autre segment (celui qui reçoit ces données) sera alors :
1000 + nombre d'octets reçus.

💡 Exemple concret :

1. Segment A (seq=1000, 20 octets de données) → envoyé au client
2. Segment B (ACK=1020, flag ACK activé) → le client répond : "J'ai bien reçu jusqu'à 1019, j'attends maintenant l'octet 1020."

Lien avec le flag ACK

- **Le flag ACK doit être activé** pour que le champ **Acknowledgment Number** soit pris en compte.
 - Si le flag ACK = 0 → le champ Acknowledgment Number **n'a pas de sens** (TCP ignore la valeur).
 - Donc :
 - **ACK = 1** → le numéro dans "Acknowledgment Number" est valide et signifie ce qui a été reçu.
 - **ACK = 0** → ce segment est juste un SYN initial ou autre, pas de confirmation.
-

Quels sont les liens entre :

Les champs WINDOW annoncés par chaque intervenant

Le débit de la connexion TCP ?

Le champ WINDOW peut-il varier au cours d'une même connexion TCP ?

Qu'est-ce que le champ Window ?

- Dans TCP, chaque segment contient un champ **Window** (16 bits).
- Il indique :

combien d'octets le destinataire est prêt à recevoir **sans accusé de réception (ACK)**.

- Autrement dit, c'est la **capacité de réception restante** de l'autre machine.

💡 Exemple :

Src: 192.168.1.5 → Dst: 192.168.1.10

Window: 14600

- 192.168.1.5 annonce qu'il peut recevoir **14 600 octets** de données avant de devoir envoyer un ACK.

Lien avec le débit de la connexion TCP

- TCP utilise **flow control (contrôle de flux)** pour ne pas saturer le récepteur.
- La taille de la **window** influence directement le **débit** :
 - **Window grande** → le serveur peut envoyer plus de données avant d'attendre un ACK → débit plus élevé.
 - **Window petite** → le serveur doit attendre souvent les ACK → débit limité.
- Formule approximative pour le débit maximum :

Débit $\approx$ Window / RTT

- **RTT** = Round-Trip Time (temps aller-retour pour un segment + ACK)

Le champ Window peut-il varier pendant la connexion ?

✅ Oui, absolument.

- TCP utilise le **flow control dynamique** :
 - Chaque participant peut ajuster sa window selon sa mémoire disponible ou sa charge.
 - Wireshark montre souvent la **Window qui change d'un segment à l'autre**.
- Exemple :
 - Au début : Window = 14600 octets

- Après avoir reçu des données et envoyé ACK : Window = 32768 octets
- Ces variations permettent à TCP de **s'adapter à la vitesse de réception** et aux conditions réseau.

4. Le payload TCP

Sélectionnez quelques trames contenant du TCP, et pour chacune, faites les relevés ci-dessous, et voyez si vous pouvez tirer des conclusions :

The screenshot shows the Wireshark interface with a packet capture of a TCP segment. The packet list at the top shows a packet from 192.168.1.37 to 35.198.80.1. The packet details pane shows the following information:

- Source Port: 5056
- Destination Port: 443
- [Stream index: 8]
- [Stream Packet Number: 1]
- [Conversation completeness: Incomplete (12)]
- [TCP Segment Len: 1]
- Sequence Number: 1 (relative sequence number)
- Sequence Number (raw): 2676569199
- [Next Sequence Number: 2 (relative sequence number)]
- Acknowledgment Number: 1 (relative ack number)
- Acknowledgment Number (raw): 2795413610
- Window: 253
- [Calculated window size: 253]
- [Window size scaling factor: -1 (unknown)]
- Checksum: 0xd61a [unverified]
- [Checksum Status: Unverified]
- Urgent Pointer: 0
- [Timestamps]
- [SEQ/ACK analysis]
- TCP payload (1-byte)
- TCP segment data (1 byte)

Red arrows point to the 'Header Length: 20 bytes (5)' and 'TCP segment data (1 byte)' fields, indicating the size of the header and the payload respectively.

Wi-Fi 3

Fichier Editer Vue Aller Capture Analyser Statistiques Telephone Wireless Outils Aide

udpZ

No.	Time	Source	Destination	Protoc	Length	Info
13	12.649088	4.187.114.131	192.168.1.37	TCP	60	443 → 5099 [RST, ACK] Seq=1 Ack=1 Win=0 Len=0
14	13.371496	192.168.1.37	192.168.1.37	TCP	55	5055 → 443 [ACK] Seq=1 Ack=1 Win=253 Len=1
15	13.388887	35.190.80.1	192.168.1.37	TCP	66	443 → 5055 [ACK] Seq=1 Ack=2 Win=1047 Len=0 SLE=1 SRE=2
16	13.878539	192.168.1.37	34.120.195.249	TCP	55	5056 → 443 [ACK] Seq=1 Ack=1 Win=253 Len=1
17	13.896359	34.120.195.249	192.168.1.37	TCP	66	443 → 5056 [ACK] Seq=1 Ack=2 Win=1047 Len=0 SLE=1 SRE=2
19	16.346949	162.159.130.234	192.168.1.37	TCP	60	443 → 5007 [ACK] Seq=1 Ack=119 Win=17 Len=0
21	16.578009	192.168.1.37	162.159.130.234	TCP	54	5007 → 443 [ACK] Seq=119 Ack=35 Win=251 Len=0
22	16.944770	162.159.130.234	192.168.1.37	TCP	68	[TCP Spurious Retransmission] 443 → 5007 [PSH, ACK] Seq=1 Ack=119 Win=17 Len=34
23	16.944869	192.168.1.37	162.159.130.234	TCP	66	[TCP Dup ACK 21#1] 5007 → 443 [ACK] Seq=119 Ack=35 Win=251 Len=0 SLE=1 SRE=35
24	17.502904	192.168.1.37	162.159.133.232	TCP	55	5096 → 443 [ACK] Seq=1 Ack=1 Win=253 Len=1
25	17.517106	162.159.133.232	192.168.1.37	TCP	60	443 → 5036 [RST] Seq=1 Win=0 Len=0
5	6.704886	104.16.103.112	192.168.1.37	TLSv1.2	92	Application Data

Frame 22: 88 bytes on wire (704 bits), 88 bytes captured (704 bits) on interface \Device\NPF_{3B21EEFE-9...}

Ethernet II, Src: SagemcomBroadband_5d:a6:e3 (f0:4d:4d:5d:a6:e3), Dst: ChongqingFug_ad:d2:b5 (d8:12:65:ad:d2:b5)

Internet Protocol Version 4, Src: 192.168.1.37, Dst: 162.159.130.234

Transmission Control Protocol, Src Port: 443, Dst Port: 5007, Seq: 1, Ack: 119, Len: 34

Source Port: 443
Destination Port: 5007
[Stream index: 9]
[Stream Packet Number: 5]
[Conversation completeness: Incomplete (12)]
[TCP Segment Len: 34]
Sequence Number: 1 (relative sequence number)
Sequence Number (raw): 2088277990
[Next Sequence Number: 35 (relative sequence number)]
Acknowledgment Number: 119 (relative ack number)
Acknowledgment number (raw): 453557665
Flags: 0x018 (PSH, ACK)
Window: 17
[Calculated window size: 17]
[Window size scaling factor: -1 (unknown)]
Checksum: 0x9b4f [unverified]
[Checksum Status: Unverified]
Urgent Pointer: 0
[Timestamps]
[SEQ/ACK analysis]
TCP payload (34 bytes)
Retransmitted TCP segment data (34 bytes)

Flags (tcp.flags), 12 bits

Paquets : 25 - Affichés : 25 (100.0%) - Perdus : 0 (0.0%)

16:16 29-09-25

Wi-Fi 3

Fichier Editer Vue Aller Capture Analyser Statistiques Telephone Wireless Outils Aide

tcp

No.	Time	Source	Destination	Protocol	Length	Info
1	0.000000	192.168.1.37	142.250.102.188	TCP	55	5182 → 5228 [ACK] Seq=1 Ack=1 Win=253 Len=1
2	0.010033	142.250.102.188	192.168.1.37	TCP	66	5228 → 5182 [ACK] Seq=1 Ack=2 Win=1047 Len=0 SLE=1 SRE=2
5	1.208514	192.168.1.37	104.16.103.112	TCP	1454	5054 → 443 [ACK] Seq=1 Ack=1 Win=4092 Len=1400 [TCP PDU reassembled in 6]
6	1.209514	192.168.1.37	104.16.103.112	TLSv1.2	1193	Application Data
8	1.213297	104.16.103.112	192.168.1.37	TCP	60	443 → 5054 [ACK] Seq=1 Ack=2540 Win=726 Len=0
10	1.239915	104.16.103.112	192.168.1.37	TLSv1.2	556	Application Data
11	1.239915	104.16.103.112	192.168.1.37	TLSv1.2	87	Application Data
12	1.239915	104.16.103.112	192.168.1.37	TLSv1.2	85	Application Data
13	1.240011	192.168.1.37	104.16.103.112	TCP	54	5054 → 443 [ACK] Seq=2540 Ack=567 Win=4090 Len=0
14	1.242466	192.168.1.37	104.16.103.112	TLSv1.2	89	Application Data
15	1.295637	104.16.103.112	192.168.1.37	TCP	60	443 → 5054 [ACK] Seq=567 Ack=2575 Win=726 Len=0
16	3.205694	192.168.1.37	104.16.102.112	TLSv1.2	96	Application Data

Frame 5: 1454 bytes on wire (11632 bits), 1454 bytes captured (11632 bits) on interface \Device\NPF_{3B21EEFE-9...}

Ethernet II, Src: ChongqingFug_ad:d2:b5 (d8:12:65:ad:d2:b5), Dst: SagemcomBroadband_5d:a6:e3 (f0:4d:4d:5d:a6:e3)

Internet Protocol Version 4, Src: 192.168.1.37, Dst: 104.16.103.112

Transmission Control Protocol, Src Port: 5054, Dst Port: 443, Seq: 1, Ack: 1, Len: 1400

Source Port: 5054
Destination Port: 443
[Stream index: 1]
[Stream Packet Number: 1]
[Conversation completeness: Incomplete (12)]
[TCP Segment Len: 1400]
Sequence Number: 1 (relative sequence number)
Sequence Number (raw): 4060245523
[Next Sequence Number: 1401 (relative sequence number)]
Acknowledgment Number: 1 (relative ack number)
Acknowledgment number (raw): 1681105227
Flags: 0x010 (ACK)
Window: 4092
[Calculated window size: 4092]
[Window size scaling factor: -1 (unknown)]
Checksum: 0xe2d3 [unverified]
[Checksum Status: Unverified]
Urgent Pointer: 0
[Timestamps]
[SEQ/ACK analysis]
TCP payload (1400 bytes)
Reassembled TCP PDU in frame 6
TCP segment data (1400 bytes)

Flags (tcp.flags), 12 bits

Paquets : 224 - Affichés : 189 (84.4%) - Perdus : 0 (0.0%)

16:23 29-09-25

Le segment TCP = en-tête TCP + données (payload).

	La taille du payload TCP	La taille de l'en-tête TCP	La taille du segment TCP.
A)	1	20	21
B)	34	20	54
C)	1400	20	1420

Sur base de vos captures, listez les types de contenus véhiculés dans vos connexions TCP.

Méthodes pour voir le contenu applicatif dans Wireshark

Regarder la colonne Protocol

- Dans la vue principale (liste des trames), tu as une colonne **Protocol**.
- Si c'est **TCP** pur → Wireshark n'a pas pu reconnaître d'application au-dessus.
- Si c'est **HTTP, TLS, FTP, SMTP, SSH, MySQL, ...** → ça veut dire que ce flux TCP transporte ce protocole applicatif.

No.	Time	Source	Destination	Protocol	Length	Info
57	6.073907	142.250.179.131	192.168.1.37	TCP	1466	80 → 41666 [ACK] Seq=1 Ack=208 Win=269568 Len=1412 [TCP PDU reassembled in 58]
58	6.073907	142.250.179.131	192.168.1.37	PKIX-C...	1094	Certificate Revocation List
59	6.074018	192.168.1.37	142.250.179.131	TCP	54	41666 → 80 [ACK] Seq=208 Ack=2453 Win=65280 Len=0
60	6.089169	192.168.1.37	142.251.39.99	HTTP	256	GET /r/gsr1.cr1 HTTP/1.1
61	6.104666	142.251.39.99	192.168.1.37	TCP	1466	80 → 41664 [ACK] Seq=1706 Ack=403 Win=269568 Len=1412 [TCP PDU reassembled in 62]
62	6.104666	142.251.39.99	192.168.1.37	PKIX-C...	1094	Certificate Revocation List
63	6.104780	192.168.1.37	142.251.39.99	TCP	54	41664 → 80 [ACK] Seq=403 Ack=4158 Win=65280 Len=0
64	6.119950	192.168.1.37	142.251.39.99	HTTP	254	GET /r/r4.cr1 HTTP/1.1
65	6.166351	142.251.39.99	192.168.1.37	PKIX-C...	1296	Certificate Revocation List
66	6.203795	192.168.1.37	81.243.1.105	TCP	66	41667 → 80 [SYN] Seq=0 Win=65535 Len=0 MSS=1460 WS=256 SACK_PERM
69	6.210431	192.168.1.37	142.251.39.99	TCP	54	41664 → 80 [ACK] Seq=603 Ack=5400 Win=64256 Len=0
70	6.215823	81.243.1.105	192.168.1.37	TCP	66	80 → 41667 [SYN, ACK] Seq=0 Ack=1 Win=64240 Len=0 MSS=1452 SACK_PERM WS=128
71	6.215950	192.168.1.37	81.243.1.105	TCP	54	41667 → 80 [ACK] Seq=1 Ack=1 Win=65280 Len=0
72	6.216358	192.168.1.37	81.243.1.105	HTTP	336	GET /msdownload/update/v3/static/trustedr/en/authrootstl.cab?c367a3bc0c9449b2 HTTP/1.1
73	6.228246	81.243.1.105	192.168.1.37	TCP	60	80 → 41667 [ACK] Seq=1 Ack=283 Win=64128 Len=0
74	6.228246	81.243.1.105	192.168.1.37	HTTP	322	HTTP/1.1 304 Not Modified
76	6.244136	192.168.1.37	81.243.1.105	HTTP	342	GET /msdownload/update/v3/static/trustedr/en/disallowedcertstl.cab?e8d5a3a48f65ca91 HTTP/1.1
77	6.255260	81.243.1.105	192.168.1.37	TCP	60	80 → 41667 [ACK] Seq=269 Ack=571 Win=64128 Len=0
78	6.255260	81.243.1.105	192.168.1.37	HTTP	322	HTTP/1.1 304 Not Modified
80	6.303374	192.168.1.37	81.243.1.105	TCP	54	41667 → 80 [ACK] Seq=571 Ack=537 Win=64768 Len=0
81	6.427047	192.168.1.37	104.16.103.112	TCP	1454	5054 → 443 [ACK] Seq=2575 Ack=567 Win=4890 Len=1400 [TCP PDU reassembled in 82]
82	6.427047	192.168.1.37	104.16.103.112	TLSv1.2	328	Application Data
83	6.439905	104.16.103.112	192.168.1.37	TCP	60	443 → 5054 [ACK] Seq=567 Ack=3975 Win=726 Len=0
84	6.439905	104.16.103.112	192.168.1.37	TCP	60	443 → 5054 [ACK] Seq=567 Ack=5149 Win=726 Len=0
85	6.454820	104.16.103.112	192.168.1.37	TLSv1.2	347	Application Data
86	6.454820	104.16.103.112	192.168.1.37	TLSv1.2	87	Application Data
87	6.454820	104.16.103.112	192.168.1.37	TLSv1.2	85	Application Data
88	6.454254	192.168.1.37	104.16.103.112	TCP	54	5054 → 443 [ACK] Seq=5149 Ack=924 Win=4895 Len=0
89	6.456997	192.168.1.37	104.16.103.112	TLSv1.2	89	Application Data
90	6.508605	104.16.103.112	192.168.1.37	TCP	60	443 → 5054 [ACK] Seq=924 Ack=5184 Win=726 Len=0
91	6.469313	192.168.1.37	20.59.87.225	TLSv1.2	155	Application Data
92	6.803662	20.59.87.225	192.168.1.37	TLSv1.2	225	Application Data
93	6.849017	192.168.1.37	20.59.87.225	TCP	54	4986 → 443 [ACK] Seq=102 Ack=172 Win=255 Len=0
94	7.193435	192.168.1.37	35.186.224.44	TLSv1.2	97	Application Data
95	7.214005	35.186.224.44	192.168.1.37	TLSv1.2	94	Application Data
96	7.255148	192.168.1.37	35.186.224.44	TCP	54	5152 → 443 [ACK] Seq=44 Ack=41 Win=255 Len=0

Frame 5: 1454 bytes on wire (11632 bits), 1454 bytes captured (11632 bits) on interface \Device\NPF...
 Ethernet II, Src: ChongqingFug...ad:d2:b5 (d8:12:65:ad:d2:b5), Dst: SagemcomBroa...5d:a6:e3 (f0:4d:d4:5d:a6:e3)
 Internet Protocol Version 4, Src: 192.168.1.37, Dst: 104.16.103.112
 Transmission Control Protocol, Src Port: 5054, Dst Port: 443, Seq: 1, Len: 1400

Flags (tcp.flags), 12 bits
 Paquets : 224 · Affichés : 189 (84.4%) · Perdus : 0 (0.0%)

Pourquoi ces applications utilisent-elles TCP ?

Ce que j'ai vu dans mes captures

- **TLSv1.2** : c'est un protocole de sécurisation des communications. Il chiffre par-dessus TCP (par exemple pour HTTPS). → Utilise **TCP** car il a besoin d'un transport fiable, ordonné et sans perte pour que le chiffrement et l'échange de clés fonctionnent correctement.
- **TCP** (comme protocole affiché tout seul) : c'est la couche de transport elle-même, donc pas un "contenu applicatif" mais le transport des autres protocoles.
- **HTTP** : protocole du web, qui échange des pages, images, API, etc. → Utilise **TCP** car les requêtes et réponses doivent arriver entières et dans le bon ordre (imagine une page web coupée en morceaux ou désordonnée).
- **PKIX-C** (Public Key Infrastructure – Certificate): ce sont des échanges liés aux certificats (ex : validation d'un certificat X.509). → Utilise **TCP** car l'échange de certificats doit être fiable et intégral (aucune perte possible).

Autres exemples classiques (même si je ne les ai pas vu)

- **SMTP** (envoi de mails), **IMAP/POP3** (récupération de mails) → utilisent TCP pour garantir que les messages arrivent complets, sans corruption.
- **FTP** (transfert de fichiers) → TCP est nécessaire pour transférer un fichier entier sans qu'il soit tronqué ou mélangé.
- **SSH** (connexion sécurisée à distance) → TCP permet une session interactive fiable.
- **HTTPS** (HTTP + TLS) → encore une fois, besoin de fiabilité pour les transactions sécurisées.

Dans quel sens sont échangées les données lors d'une connexion TCP ?

1. TCP est full-duplex

- Une connexion TCP permet **l'envoi et la réception de données simultanément**.
- Chaque extrémité (client et serveur) peut envoyer des données indépendamment de l'autre.

2. Exemple concret : client ↔ serveur HTTP

1. Établissement de connexion (3-way handshake)

- Client → Serveur : SYN
- Serveur → Client : SYN + ACK
- Client → Serveur : ACK

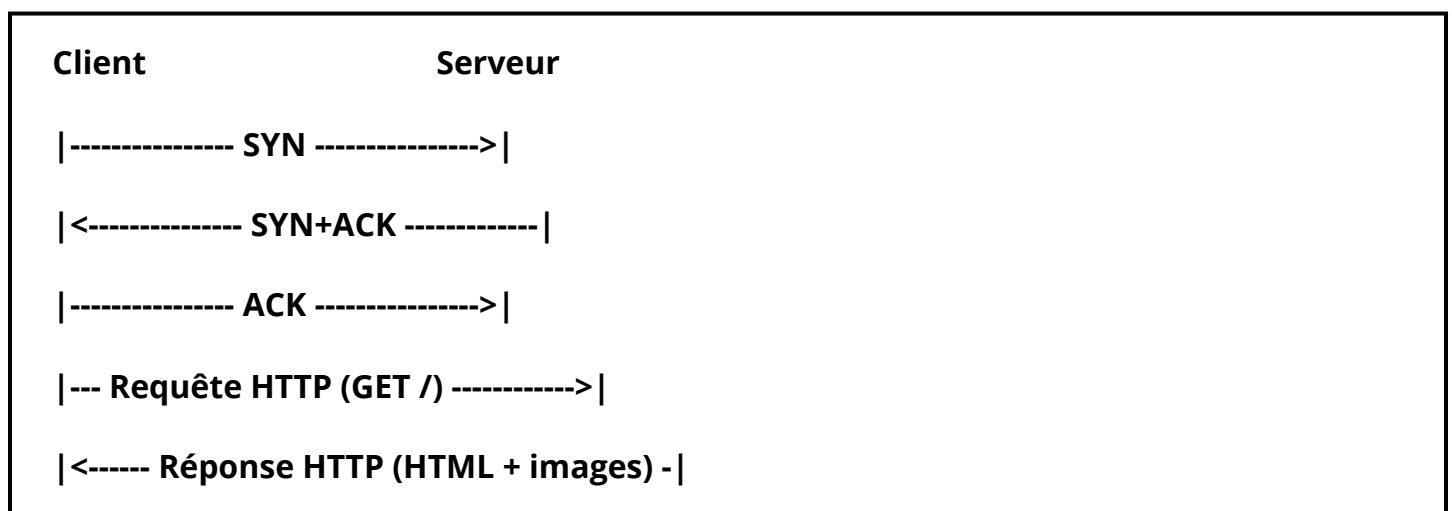
2. Transmission des données

- Client → Serveur : requête HTTP (GET /index.html)
- Serveur → Client : réponse HTTP (page web, images...)
- **Ici, on voit les deux sens :**
 - Client → Serveur = requêtes
 - Serveur → Client = réponses

3. Remarques importantes

- TCP utilise **numéros de séquence et ACK** pour chaque sens de transmission.
 - Chaque côté garde sa propre numérotation.
- La communication est donc **bidirectionnelle**, mais chaque flux est indépendant et fiable.

4. Schéma simplifié



Tu vois que les **données circulent dans les deux sens** après le handshake.

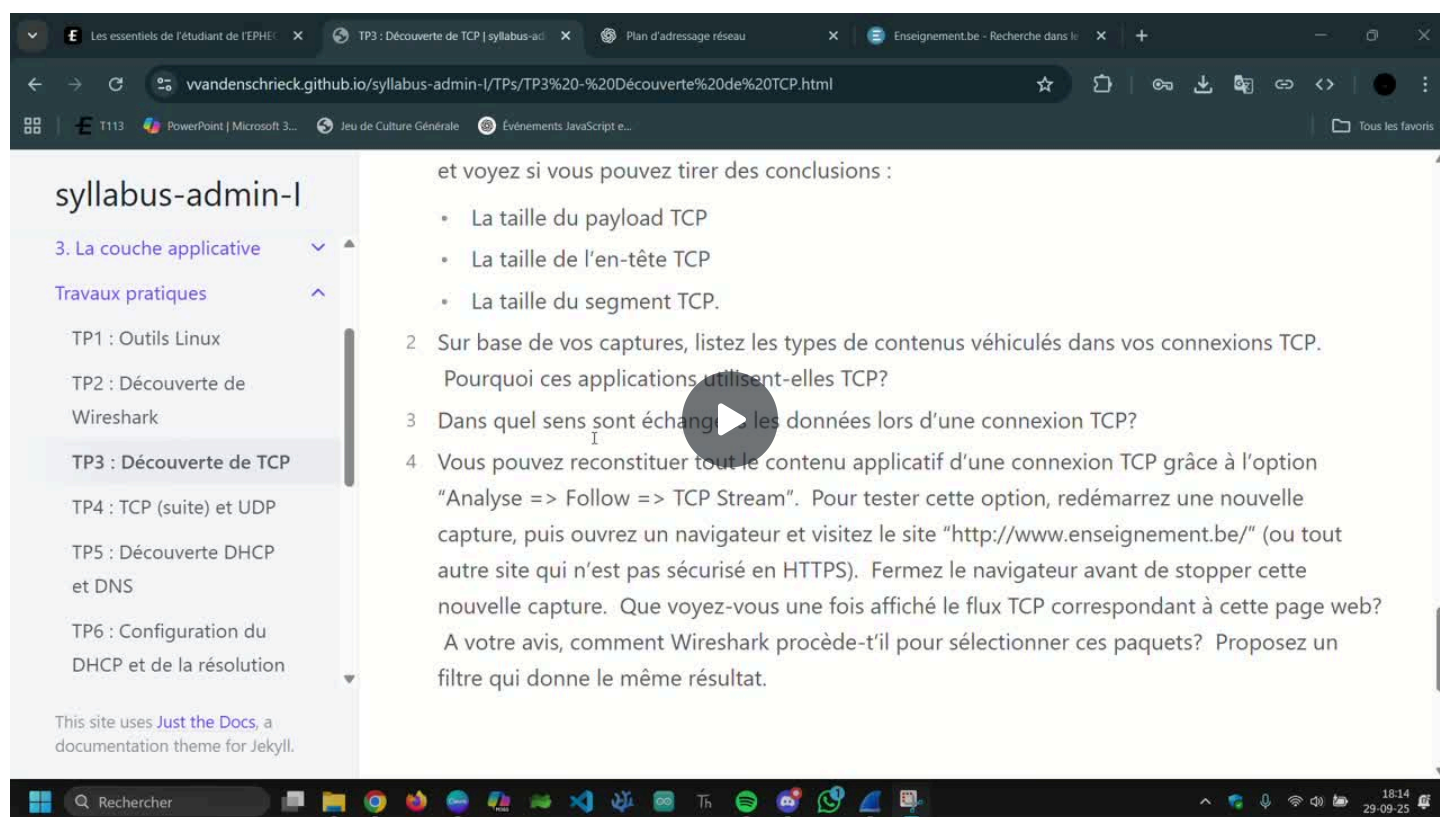
Vous pouvez reconstituer tout le contenu applicatif d'une connexion TCP grâce à l'option "Analyse => Follow => TCP Stream".

Pour tester cette option, redémarrez une nouvelle capture, puis ouvrez un navigateur et visitez le site "http://www.enseignement.be/" (ou tout autre site qui n'est pas sécurisé en HTTPS).

Fermez le navigateur avant de stopper cette nouvelle capture. Que voyez-vous une fois affiché le flux TCP correspondant à cette page web ?

A votre avis, comment Wireshark procède-t'il pour sélectionner ces paquets ?

Proposez un filtre qui donne le même résultat.



Ce que vous verrez dans le flux TCP

Lorsque tu fais **Analyze → Follow → TCP Stream** sur la capture HTTP de ton site :

Tu verras **tout le dialogue entre ton PC et le serveur web** pour cette connexion TCP.

Typiquement :

Requête HTTP du client (ton PC)

GET / HTTP/1.1

Host: www.enseignement.be

User-Agent: ...

Accept: ...

Réponse HTTP du serveur (77.246.241.124)

HTTP/1.1 200 OK

Content-Type: text/html

Content-Length: xxxx

...

<html>...</html>

Les requêtes et réponses sont affichées **dans l'ordre**, même si elles ont été fragmentées en plusieurs segments TCP.

Comment Wireshark sélectionne ces paquets

Wireshark prend **tous les paquets appartenant à cette connexion TCP** en utilisant la **quintuple** :

(IP source, IP destination, Port source, Port destination, TCP)

- **Dans ton cas :**
 - **IP source : 192.168.1.37**
 - **IP destination : 77.246.241.124**
 - **Port source : 17195**
 - **Port destination : 80**
- Tous les paquets qui partagent ces paramètres sont affichés dans le flux reconstitué.

Filtre Wireshark équivalent

Pour voir **les mêmes paquets que Follow TCP Stream** dans la liste principale :

tcp && ip.addr== 192.168.1.37 && ip.addr==77.246.241.124 && tcp.port==17195

Tu verras **tous les segments TCP de cette connexion** dans la liste des paquets.

Alternative simple (si tu ne veux filtrer que sur le port serveur HTTP) :

tcp.port == 80

Cela montre toutes les connexions HTTP sur le port 80, mais pas uniquement celle de ton flux précis.

5. Ouverture et fermeture de connexion TCP

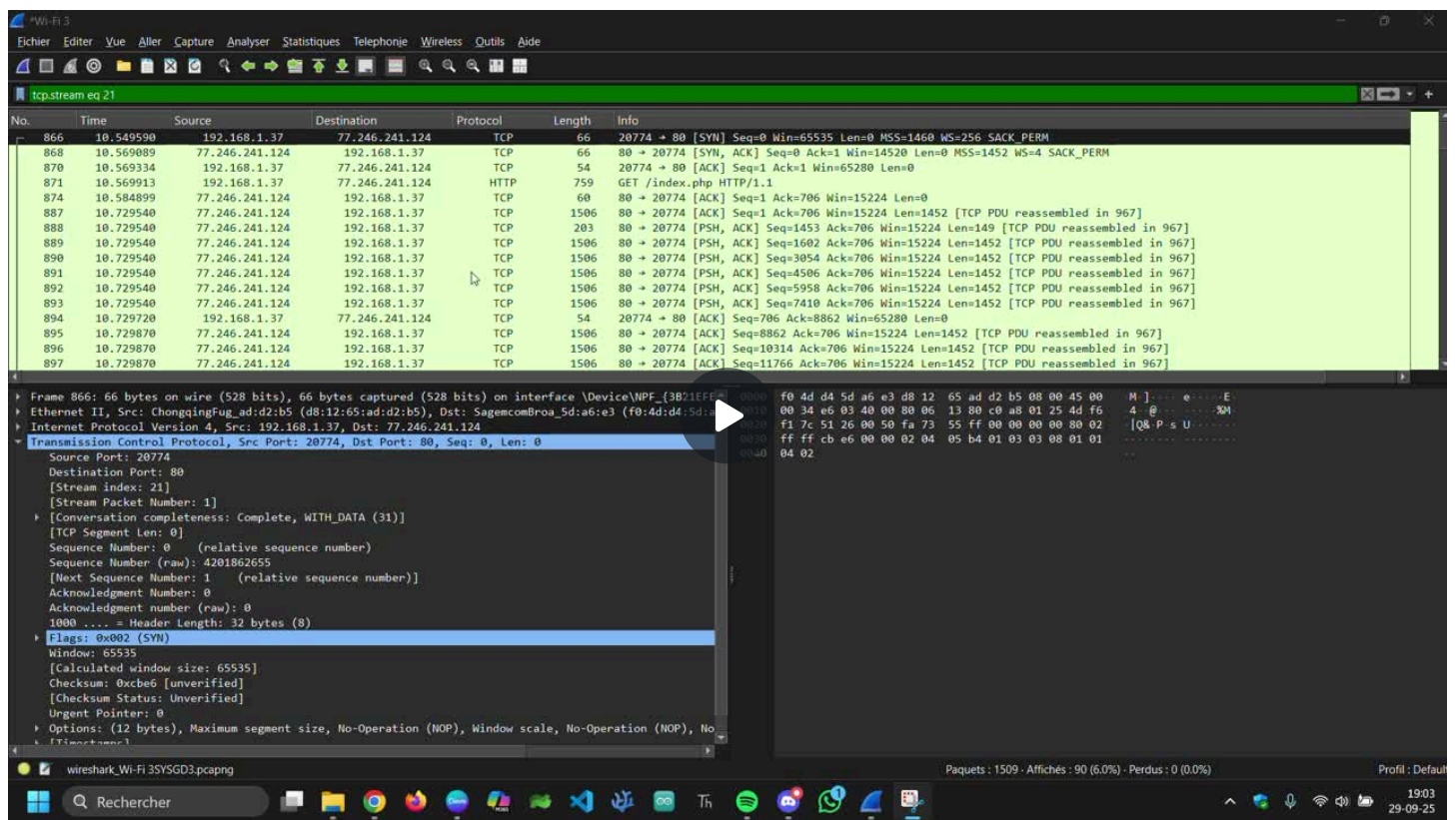
Sur la trace précédente, quels sont les flags des premiers segments échangés ?
Essayez de retrouver ce “pattern” dans d’autres connexions TCP.

Sur ces 3 messages, quels flags sont utilisés par l’initiateur de la connexion ?

Et par le destinataire ?

Pourquoi ?

Quelles sont les valeurs des numéros de séquence et des numéros d’acquis de ces messages initiaux ? (faites attention aux numéros relatifs de Wireshark)



Les flags des premiers segments

Le **handshake TCP standard** se fait en **3 étapes (3-way handshake)** :

Étape	Segment TCP envoyé par	Flags TCP
1	Client → Serveur	SYN (synchronize)
2	Serveur → Client	SYN + ACK (synchronize + acknowledge)
3	Client → Serveur	ACK (acknowledge)

- **SYN** : indique qu'on veut établir une connexion.
- **ACK** : confirme la réception d'un segment précédent.

Donc sur ta trace précédente, si tu regardes les **premiers segments TCP** :

- Le client (ton PC, 192.168.1.37) envoie **SYN**.
- Le serveur (77.246.241.124) répond **SYN+ACK**.
- Le client termine avec **ACK**.

C'est exactement le "pattern" typique des **débuts de connexion TCP**.

Retrouver ce pattern dans d'autres connexions TCP

Ce pattern **SYN → SYN+ACK → ACK** est **identique pour toutes les connexions TCP**, quelle que soit l'application (HTTP, FTP, SMTP, etc.).

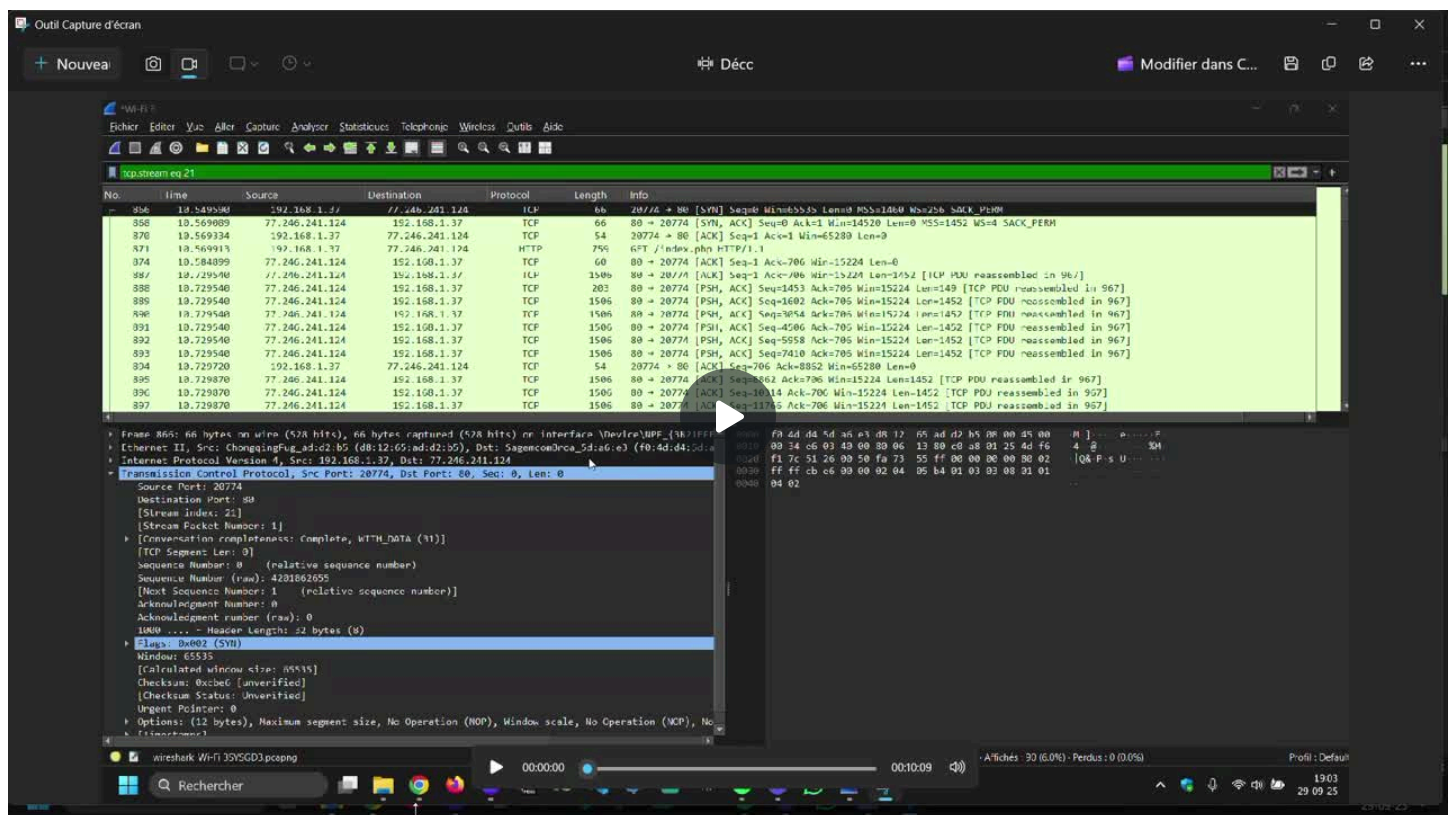
Pour le vérifier dans Wireshark :

1. Mets un **filtre TCP** pour l'IP ou le port que tu veux analyser.
Exemple : tcp && ip.addr==77.246.241.124
2. Cherche les segments avec **Flag = SYN** ou **SYN+ACK**.
3. Tu verras toujours ce pattern de trois paquets au début de la connexion.

Mêmes questions pour les segments de fin de connexion.

Il existe deux types de fermeture, les voyez-vous dans vos traces ?

Dans chaque cas, quels flags sont impliqués ?



Les types de fermeture TCP

Il y a **2 façons de terminer une connexion** :

Fermeture normale (gracieuse, "4-way handshake")

C'est la plus fréquente. Chaque côté ferme **proprement** sa connexion :

1. **Client → Serveur** : envoie un segment avec **FIN + ACK** (je n'ai plus de données à envoyer).
2. **Serveur → Client** : répond par **ACK** (j'ai bien reçu ton FIN).
3. **Serveur → Client** : envoie à son tour **FIN + ACK** (je n'ai plus de données non plus).
4. **Client → Serveur** : répond par **ACK** final (connexion fermée).

👉 Donc flags utilisés : **FIN et ACK** (pas SYN).

Fermeture brutale (par Reset, RST)

Parfois, une connexion est coupée directement (exemple : service arrêté brutalement, socket fermée de force) :

- **Un côté envoie un segment avec le flag RST** (reset).
- L'autre n'a pas à répondre, la connexion est interrompue immédiatement.

👉 Flag utilisé : **RST** (reset).

Ce que tu devrais voir dans tes traces

Dans la plupart des **HTTP non sécurisés (port 80)** :

→ Fermeture normale avec **FIN/ACK** des deux côtés.

Si tu observes une connexion coupée brutalement (par exemple à cause d'un timeout ou d'une erreur) :

→ Fermeture par **RST**.